

Project Report: Cyber Crime Complaint Management System

1. Introduction

The **Cyber Crime Complaint Management System** is a lightweight, efficient software solution designed to streamline the reporting, tracking, and management of cybercrime incidents. The application operates as a **Command-Line Interface (CLI)** program, offering an interactive, terminal-based experience.

This streamlined architecture provides a fast, text-based platform for registering cybercrime complaints (such as Phishing, Financial Fraud, and Identity Theft) while allowing administrators to track and update the lifecycle of each case directly from their console.

2. Objectives

- Efficiency:** To provide a fast, terminal-based interface for managing cybercrime records without the overhead of a web server or graphical UI.
- Case Tracking:** To enable the generation of unique, standardized complaint IDs (e.g., CYB1001) for seamless case tracking.
- Data Persistence:** To utilize a relational database (H2) for robust data integrity and querying.
- Analytics:** To provide instant system statistics, priority cases, and resolution metrics directly within the console.

3. Technology Stack

The application employs a robust Java backend optimized for CLI execution:

- Framework:** Spring Boot 3.3.x (Java 17) using `CommandLineRunner`
- Data Access:** Spring Data JPA / Hibernate
- Database:** H2 In-Memory Relational Database (Configured for easy testing and schema auto-generation)
- Build Tool:** Maven Wrapper (`mvnw`)

4. System Architecture

The system follows a streamlined service-oriented architecture:

- CLI Runner (`CliRunner.java`):** Intercepts the Spring Boot startup lifecycle to launch an interactive `java.util.Scanner` loop, providing a text-based menu to the user.
- Service Layer (`ComplaintService.java`):** Encapsulates business logic, including custom ID generation and statistical calculations for system reports.
- Repository Layer (`ComplaintRepository.java`):** Extends `JpaRepository` to interface directly with the H2 Database.
- Model Layer (`Complaint.java`):** Defines the JPA Entity mapped to the database table.

5. Key Features & Modules

5.1 System Report Generator

A reporting module that provides a bird's-eye view of the system. It calculates and prints:

- Total Complaints Registered
- Open / Pending Complaints
- Resolved Cases
- High-Priority Incidents
- The most frequently reported crime category

5.2 Complaint Registration

An interactive console prompt allowing victims to submit details of the incident.

- Fields captured:** Name, Contact Number, Crime Type, Priority Level, Financial Loss, and Incident Description.
- Automation:** Upon submission, the system automatically tags the complaint with a `REGISTERED` status, records the creation timestamp, and auto-generates a unique `CYBxxxx` ID.

5.3 Search and Lifecycle Management

Administrators can search for any case using its unique Complaint ID via the menu.

- Data Retrieval:** Instantly fetches and formats the full details of the case in the terminal.
- Status Updates:** Allows authorized personnel to update the case status through its lifecycle (`REGISTERED` → `UNDER REVIEW` → `ASSIGNED` → `UNDER INVESTIGATION` → `RESOLVED` → `CLOSED`).

5.4 Demo Dataset Integration

To facilitate immediate demonstration and testing, the system boots with a predefined `data.sql` script. This injects 10 realistic, diverse cybercrime cases into the database upon startup, immediately populating the reports and search indexes.

6. Setup and Execution

The project is entirely self-contained and requires no external database server installations.

- Ensure Java 17+ is installed.
- Navigate to the project root directory.
- Run the application using the Maven wrapper:

```
./mvnw spring-boot:run
```

- Follow the interactive prompts in the terminal to navigate the system.

7. Conclusion and Future Enhancements

The Cyber Crime Complaint Management System provides a highly reliable, text-based tool for incident management. By leveraging Spring Boot's dependency injection and JPA capabilities without the overhead of a web server, it serves as a lightweight, lightning-fast utility.

Proposed Future Enhancements:

- **Security & Authentication:** Require a password or token to access the administrative features of the CLI (like updating status).
- **Export Functionality:** Add a menu option to export the system report to a CSV or PDF file.
- **Persistent Database:** Swap the H2 in-memory database for a file-based SQLite database or PostgreSQL for persistent, long-term deployment.